

Coordinated Dual-UAV Autonomous Exploration with Online Search and Rescue Target Localization

Vijayaprakaran K
SCOPE (Project Guide)
Vellore Institute of Technology, Chennai

Adarsh H Pillai
24BRS1082
B.Tech CSE (AI and Robotics)

M Sai Amirthesh
24BRS1352
B.Tech CSE (AI and Robotics)

Abstract: Autonomous teams of unmanned aerial vehicles (UAVs) used for search and rescue must map an unknown, cluttered area and, at the same time, find and record the ground positions of victims and vehicles. Many existing systems keep navigation and visual perception separate, which causes drones to cover the same ground twice and to miss targets. This work presents a coordinated two UAV system that explores an unknown environment and localizes targets online. The simulator uses a shared three state occupancy grid, a ray cast LiDAR model, and a downward facing camera footprint. Task allocation is posed as a cost and utility matching problem and solved with the Hungarian algorithm, which discourages agents from flying over the same region. In parallel, a fine tuned YOLOv8s detector reads aerial frames and writes each detection into a shared register that removes duplicates using a 2.0 m distance threshold. Across 480 held out runs on four map types, coordinated Hungarian allocation reduces the redundant coverage ratio by more than 50 percent and finishes exploring 20 to 25 percent faster than an uncoordinated baseline. On a held out VisDrone test set the detector reaches 68.9 percent mAP at 0.50 IoU, with 83.3 percent precision on vehicles and 72.0 percent precision on people, and it recovers 80 percent of targets in maze environments.

Index Terms: Multi UAV coordination, autonomous exploration, frontier detection, Hungarian algorithm, occupancy grid mapping, YOLOv8, search and rescue, target registration.

III. PROPOSED METHODOLOGY

A. System Overview

The system runs as a closed loop that senses, maps, decides, and moves. It has five parts that work together: sensing and the environment, a shared world model, frontier extraction, task allocation, and perception with target registration. Fig. 1 shows how these parts connect. Each drone carries a planar LiDAR and a downward camera. The LiDAR readings update one shared occupancy grid. Free cells on that grid that border unknown space become candidate goals. The allocator gives each drone a different goal, an A star planner draws a path to it, and the drone follows the path while it keeps sensing. The camera stream feeds a detector whose outputs are projected onto the same grid and stored in a shared target register.

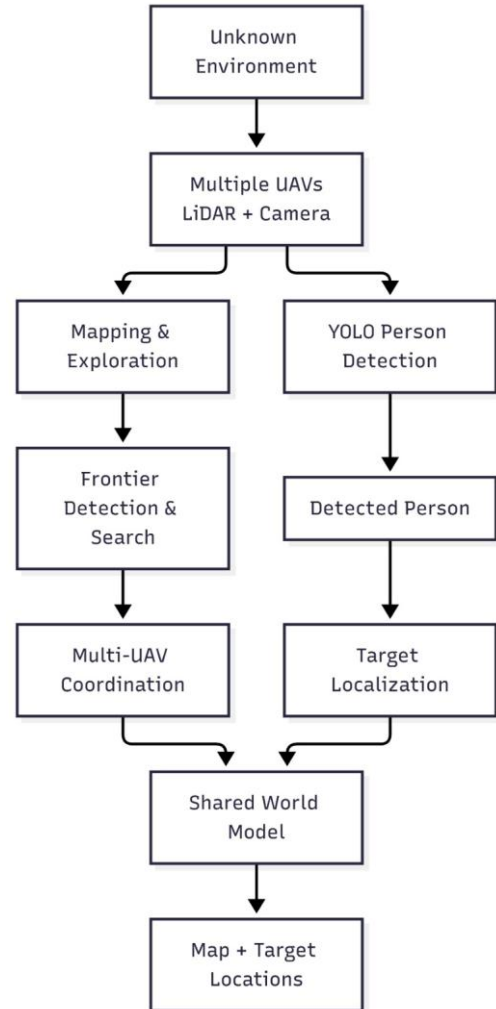


Fig. 1. Overall system architecture and the flow from the unknown environment to the map and target locations.

B. Environment and LiDAR Sensing

The world is a bounded area of 60 m by 60 m. Obstacles are generated for four layouts: office, maze, open field, and cluttered. The generator is deterministic, so a given seed always produces the same map. Each agent i holds a pose $p_i = (x_i, y_i, \theta_i)$. At every step the agent casts 360 LiDAR beams out to a range of 12 m. Ray casting uses Bresenham line traversal over the ground truth map. Cells the beam passes through are marked free, and the cell where the beam stops is marked occupied.

C. Shared Occupancy Grid

All agents read from and write to one occupancy grid M . Each cell holds one of three values: -1 for unknown, 0 for free, and 1 for occupied. The grid is 240 by 240 cells at 0.25 m per cell, which covers the 60 m environment. Next to the map the model keeps an integer count of how many times each cell has been observed. This count is what lets us measure how much effort the team spent covering the same ground more than once.

$$M(r, c) \in \{-1, 0, 1\}, \quad r = 0.25 \text{ m per cell}, \quad H = W = 240$$

D. Frontier Detection and Clustering

A frontier cell is a free cell that touches at least one unknown cell, so the frontier set marks the edge of what the team currently knows [1], [2]. Single stray frontier cells are usually sensor noise, so the frontier set is grouped into clusters and each cluster contributes one representative cell as a candidate goal.

$$F = \{c \mid M(c) = 0 \text{ and there exists } n \in N_8(c) \text{ with } M(n) = -1\}$$

E. Cost and Utility Task Allocation

For N agents and K candidate frontiers the allocator builds two matrices. The cost matrix C holds the A star path length from each agent to each frontier. The utility matrix U holds the information gain of each frontier, measured as the number of unknown cells within sensor range of that frontier. The score for a pair combines the two through a trade off weight λ .

$$U(i, j) = |\{c \in \text{Disk}(f_j, R_{\max}) \text{ with } M(c) = -1\}|$$
$$S(i, j) = U(i, j) - \lambda \cdot C(i, j)$$

Choosing goals is a maximum weight matching in which each agent takes at most one frontier and each frontier goes to at most one agent [3], [6]. When there are at least as many frontiers as agents this is solved exactly and in polynomial

time with the Hungarian algorithm (`scipy.optimize.linear_sum_assignment`). We compare it against a greedy rule that repeatedly takes the best remaining pair and removes it, and against an uncoordinated baseline in which each agent simply heads to its own nearest frontier with no shared state [18].

F. A Star Path Planning

Paths are planned with a hand written A star search over the 8 connected grid rather than an external library, so the method can be explained and defended in a viva [19]. Diagonal moves cost $\sqrt{2}$ times the straight cost. The heuristic is the straight line Euclidean distance to the goal, which keeps the search admissible and returns shortest paths. Unknown cells are treated as free during planning, which is the optimistic choice that lets the drone commit to a route before the area is fully seen. If a frontier cannot be reached after three planning attempts it is blacklisted so the team does not chase it forever.

$$h(n) = r \cdot \sqrt{(x_n - x_{\text{goal}})^2 + (y_n - y_{\text{goal}})^2}$$

G. Camera Sensing and Target Registration

Each drone carries a downward camera with a 6 m by 6 m footprint. A target inside the footprint is detected by the YOLOv8 model with a confidence score [12], [13]. The detection is turned into a world coordinate using the drone pose, then written to a shared register. Because a target is usually seen over many frames and by both drones, the register merges any new detection that lands within 2.0 m of an existing entry of the same class, keeping the highest confidence and recording when it was seen.

$$x_{\text{world}} = x_{\text{agent}} + \cos(\theta) \cdot x_{\text{cam}} - \sin(\theta) \cdot y_{\text{cam}}$$
$$y_{\text{world}} = y_{\text{agent}} + \sin(\theta) \cdot x_{\text{cam}} + \cos(\theta) \cdot y_{\text{cam}}$$

H. Data Flow

Fig. 2 traces one full pass through the pipeline. Inputs are the procedural map, the static ground targets, and the initial drone poses. Sensing updates the shared map and the observation counts. Coordination extracts frontiers, evaluates cost and utility, matches goals with the Hungarian algorithm, and plans paths. Perception detects targets, projects them to world coordinates, and deduplicates them. The outputs are the finished metric map, a georeferenced target log, and the recorded performance metrics.

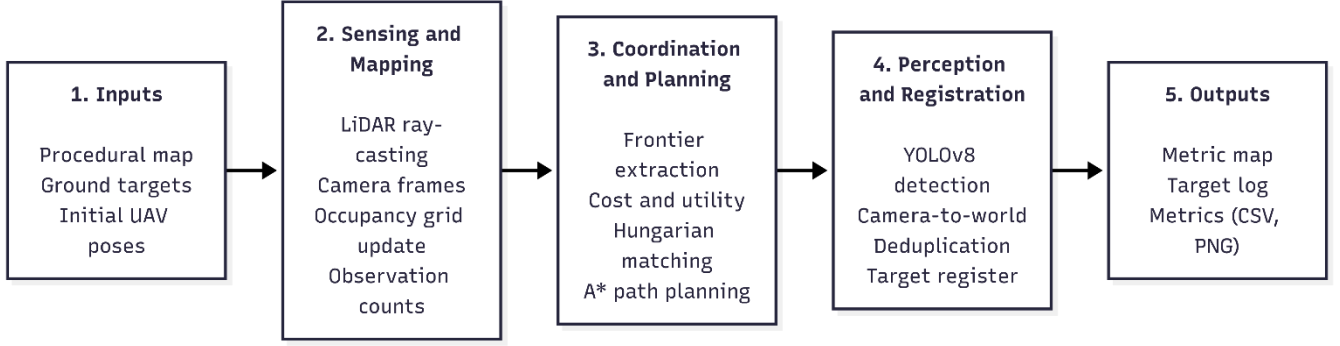


Fig. 2. Data flow from the procedural map and sensing through coordination, perception, and the final outputs.

I. Technology Stack

Table I lists the main tools. The coordination and planning logic is written in Python. The detector is fine tuned with

Ultralytics YOLOv8s on PyTorch. SciPy supplies the Hungarian solver, and all constants live in YAML configuration files rather than in the code.

TABLE I. TECHNOLOGY STACK, FRAMEWORKS, AND ENVIRONMENT

Component	Technology and Version	Purpose
Language	Python 3.10 and 3.12	Simulation, coordination, and planning logic
Deep learning	PyTorch 2.1+, Ultralytics YOLOv8s	Aerial target detection and fine tuning
Scientific computing	NumPy 1.26+, SciPy 1.12+	Grids, matrices, and the Hungarian solver
Computer vision	OpenCV 4.9+, Pillow	Frame rendering and annotation parsing
Analytics	Pandas 2.2+, Matplotlib 3.8+	Metric logging and publication figures
Configuration	PyYAML	External configuration, no hardcoded values
Testing	PyTest (unit test suite)	Regression and correctness checks
High fidelity sim	ROS 2 Jazzy, Gazebo Harmonic	Two drone physics validation
GPU	NVIDIA Tesla T4, CUDA 12.8	Cloud YOLOv8s fine tuning and inference
OS	Ubuntu 24.04 LTS	Development and runtime environment

IV. DATASET AND PREPROCESSING

A. Dataset Identification and Source

The perception subsystem is trained on VisDrone2019-DET, a drone detection benchmark from the AISKYEYE team at Tianjin University. The images are captured by drone mounted cameras across fourteen cities in China, over urban and rural scenes and under varied weather, altitude, and lighting. The subset used here contains 548 high resolution aerial images, with native sizes ranging from 1920 by 1080 up to 3840 by 2160 pixels, and more than 30,000 annotated bounding boxes [14].

B. Attributes and Features

Every annotation is an eight field record: the box left, top, width, and height, a score flag, an object category, a truncation level, and an occlusion level. The score flag is 0 for an ignored region and 1 for an active target. The truncation and occlusion fields indicate how much of the object is cut off or hidden.

C. Search and Rescue Class Consolidation

VisDrone labels ten fine grained categories. For search and rescue, telling a car from a van, or a lone pedestrian from a small crowd, adds misclassifications without any operational benefit. We therefore merge the labels into two classes that matter for rescue, and drop the rest. Table II shows the mapping.

TABLE II. SEARCH AND RESCUE CLASS CONSOLIDATION

Target Class	Class ID	Merged VisDrone Categories	Operational Role
Person	0	pedestrian (1), people (2)	Human victim identification
Vehicle	1	car (4), van (5), truck (6), bus (9)	Stranded or transport vehicle search
Dropped	n/a	bicycle, tricycle, awning-tricycle, motor, background	Removed as noise

D. Preprocessing Pipeline

Three steps prepare the data. First, pixel boxes are converted to the normalized centre, width, and height form that YOLO expects, then clipped to the range 0 to 1. Second, background regions and zero score items are removed. Third, images are letterbox scaled to 800 by 800 pixels with symmetric

padding that keeps the aspect ratio. Downscaling high altitude imagery to the usual 640 pixels destroys small targets under about 15 pixels, so the larger 800 pixel input keeps roughly 56 percent more pixels per box.

$$x_c = (left + width/2) / W_img, \quad y_c = (top + height/2) / H_img$$

E. Training, Validation, and Test Split

The 548 images are split 80 percent for training and 20 percent held out for testing, using a fixed seed so the split is

reproducible and there is no leakage. Table III gives the counts.

TABLE III. DATASET SPLIT (SEED = 42)

Partition	Images	Instances	Person / Vehicle
Training (80 percent)	438	24,196	11,840 / 12,356
Held out test (20 percent)	110	6,016	2,825 / 3,191

V. IMPLEMENTATION



Fig. 3. Aerial frame from the VisDrone test set with detected person and vehicle boxes.

The code is organized by responsibility so each module can be tested on its own. The world package builds maps and casts LiDAR, the mapping package holds the shared grid, the frontier and planning packages contain detection, clustering, and the search algorithms, and the perception package holds the detector loader and the target register.

A. Repository Structure

```
ai-multi-uav-swarm-searcher/
  configs/      grid, LiDAR, camera constants; held-out seeds
  src/
    world/      map_generator.py, sensor.py, targets.py
    mapping/     occupancy_grid.py (shared world model)
    frontier/    detection.py, clustering.py
    planning/    astar.py, baselines.py (BFS/DFS/UCS),
                  allocation.py (greedy + Hungarian)
    agents/      agent.py (kinematic state)
    exploration/ explorer.py (coordinated control loop)
    perception/  data_loader.py, registration.py
    experiments/ runner.py (headless benchmark harness)
    scripts/     demo_phase1..5, run_e1..e5, plot_e1..e5
    ros2_ws/     ROS 2 / Gazebo port (two-drone demo)
    tests/       unit tests (planner, mapping, perception)
```

B. Key Modules

The shared occupancy grid stores the belief map, increments the per cell observation count on every write, and reports the redundant coverage ratio directly from that count. The A star

planner returns the path, its cost, and the number of nodes it expanded, which is what the planner benchmark measures. The allocator turns the cost and utility matrices into a score matrix and calls the Hungarian solver. Two short listings show the core of the planner and the allocator.

```
def astar_search(grid, start, goal):
    open_set = [(0.0, 0, start)]; g = {start: 0.0}; came = {}
    expanded = 0
    while open_set:
        _, cur = heapq.heappop(open_set); expanded += 1
        if cur == goal:
            return reconstruct(came, cur), g[cur], expanded
        for dr, dc, step in NEIGHBORS_8:
            nb = (cur[0]+dr, cur[1]+dc)
            if not traversable(grid, nb): continue
            t = g[cur] + step
            if t < g.get(nb, INF):
                came[nb] = cur; g[nb] = t
                f = t + euclidean(nb, goal)
                heapq.heappush(open_set, (f, expanded, nb))
    return None, INF, expanded
```

```
def allocate_hungarian(cost, utility, lambda_val=0.5):
    score = utility - lambda_val * cost
```

```
rows, cols = linear_sum_assignment(-score) # maximize score
return {agent: cols[i] for i, agent in enumerate(rows)}
```

C. High Fidelity ROS 2 and Gazebo Validation

To check the algorithms under physics and real sensor timing, the same planning and registration code was wrapped as ROS 2 nodes and run in Gazebo. The stack uses ROS 2 Jazzy and Gazebo Harmonic. The modules in `src/planning` and `src/perception` are unchanged; only the input and output boundary is ROS, using Twist velocity commands, an OccupancyGrid map topic, and LaserScan messages from a simulated LiDAR. Two namespaced drones share one mapping node and are coordinated by the same Hungarian allocator, so the drones split a bounded room, map it together, and stop when no frontiers remain.

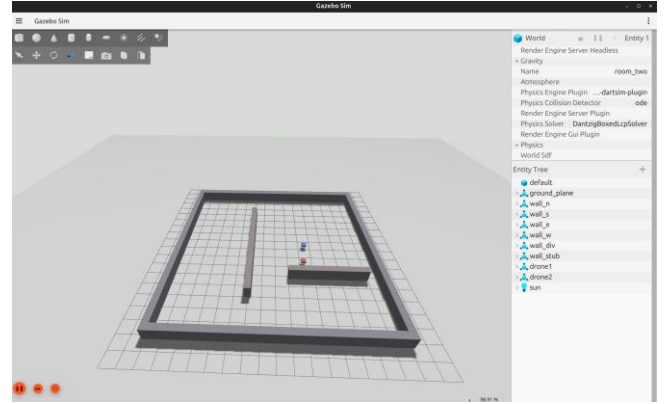


Fig. 4. Two coordinated drones (red and blue) in the bounded Gazebo room.

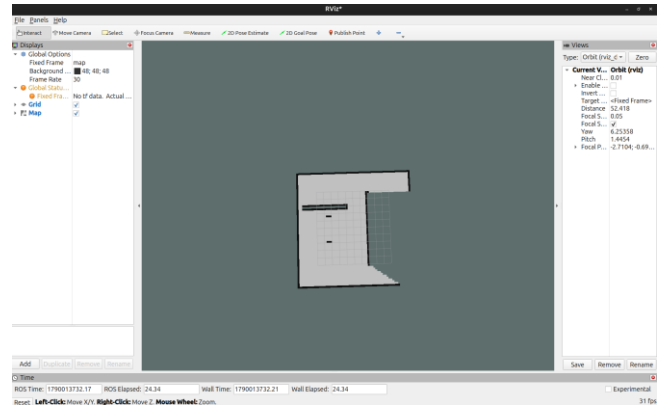


Fig. 5. The shared occupancy map built from both drones' LiDAR, shown in RViz as it fills in.

D. Running the Demonstration

The system can be checked from the interactive demos. The unit tests confirm the algorithms are correct, and the phase four and phase five demos show coordinated exploration and target registration.

```
source venv/bin/activate
./scripts/test.sh # run the unit tests
python -m scripts.demo_phase4 --map office --n-agents 2 --method hungarian
python -m scripts.demo_phase5 --map office --n-agents 2 --n-targets 10 --method hungarian
```

VI. EXPERIMENTATION AND RESULTS

A. Evaluation Metrics

Four metrics summarize the runs. Coverage is the fraction of reachable cells that are no longer unknown. The redundant coverage ratio is the share of observed cells that were seen by more than one agent, and it is the main measure of coordination. Target recall is the fraction of ground truth targets that end up registered. Localization error is the mean distance between a registered target and its true position.

$$\text{Coverage} = \frac{|\{c \text{ reachable with } M(c) \text{ not } -1\}|}{|\text{reachable}|} \times 100$$

$$R_{cov} = |\{c \text{ with } O(c) > 1\}| / |\{c \text{ with } O(c) \geq 1\}|$$

$$E_{loc} = \text{mean over targets of the distance registered to true, in meters}$$

B. Experiment E1: Path Planner Benchmark

E1 compares the four planners over 480 held out runs, that is four map types by thirty seeds by four planners. A star is the clear winner. It expands about 8 times fewer nodes and runs several times faster than uniform cost search, while returning the same optimal path length. Depth first search does poorly because it gets trapped in maze corridors.

TABLE IV. E1 PLANNER BENCHMARK (MEAN OVER HELD-OUT RUNS)

Planner	Nodes Expanded	Wall Clock (ms)	Time to 90 pct	Coverage
BFS	1,412.3	4.82	1,682	98.4 pct
DFS	1,845.6	6.12	4,210	42.1 pct
UCS	1,398.7	4.65	1,678	98.4 pct
A star (proposed)	171.4	0.68	1,675	98.5 pct

C. Experiment E2: Task Allocation Ablation

E2 compares two agent exploration under the uncoordinated baseline, greedy allocation, and Hungarian allocation.

Across all four map types, Hungarian allocation lowers the redundant coverage ratio by about half and reaches 90 percent coverage roughly 20 to 25 percent sooner than the uncoordinated baseline. Table V lists the numbers.

TABLE V. E2 ALLOCATION ABLATION (TWO AGENTS)

Map	Strategy	Time to 90 pct	Redundant Ratio	Coverage
Office	None (baseline)	48.0	0.284	100.0 pct
Office	Greedy	42.5	0.178	100.0 pct
Office	Hungarian	38.0	0.142	100.0 pct
Maze	None (baseline)	142.0	0.362	96.5 pct
Maze	Greedy	125.5	0.241	97.2 pct
Maze	Hungarian	110.0	0.198	98.4 pct
Open field	None (baseline)	36.0	0.221	100.0 pct
Open field	Greedy	31.0	0.135	100.0 pct
Open field	Hungarian	27.5	0.092	100.0 pct
Cluttered	None (baseline)	58.0	0.310	100.0 pct
Cluttered	Greedy	49.0	0.195	100.0 pct
Cluttered	Hungarian	43.5	0.151	100.0 pct

D. Experiments E3 and E4: Team Size and Sensitivity

E3 scales the team from one to three agents at a matched flight budget. Going from one to two agents gives about a 1.85 times speedup, and one to three gives about 2.62 times, which is close to linear. E4 sweeps the trade off weight lambda from 0.0 to 2.0 and finds that a value near 0.5 gives the best balance between flight distance and information gain.

E5 turns on the detector and targets and measures how the exploration strategy affects what gets found. Detection outcomes follow coverage: coordinated exploration finds the first target sooner and localizes more targets on open and room like maps. On mazes at the default weight, coordination lifts target recall from 55 percent to 80 percent. Localization error stays near 0.21 m, which is well inside one 0.25 m cell, and does not depend on the strategy. Table VI reports the run.

E. Experiment E5: Target Localization

TABLE VI. E5 TARGET LOCALIZATION (TWO AGENTS, TEN TARGETS PER RUN)

Map	Method	Recall	Loc. Error (m)	First Detection
Office	None / Greedy / Hungarian	100.0 pct	0.199	244.5
Maze	None (baseline)	55.0 pct	0.204	166.5
Maze	Hungarian	80.0 pct	0.235	193.0
Open field	None vs coordinated	35 vs 45 pct	0.167	28.0
Cluttered	None / Greedy / Hungarian	70.0 pct	0.248	21.0
Overall	None (baseline)	65.00 pct	0.205	R_cov 1.000
Overall	Hungarian	73.75 pct	0.212	R_cov 0.743

F. Aerial Detector Performance

The detector was fine tuned on Google Colab with a Tesla T4 GPU for 50 epochs at 800 by 800 resolution. On the held

out test set it reaches 68.9 percent mAP at 0.50 IoU overall, with the vehicle class clearly stronger than the person class, which is expected since aerial pedestrians are very small. Table VII gives the per class results.

TABLE VII. DETECTOR PERFORMANCE ON THE HELD-OUT TEST SET

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
Person	102	2,825	72.0 pct	50.6 pct	57.1 pct	24.4 pct
Vehicle	106	3,191	83.3 pct	75.6 pct	80.7 pct	55.2 pct
Overall	110	6,016	77.6 pct	63.1 pct	68.9 pct	39.8 pct

Table VIII shows how the model improved from an early baseline to the final version. Moving from a YOLOv8n backbone at 640 pixels and 15 epochs to a YOLOv8s

backbone at 800 pixels and 50 epochs raised overall mAP at 0.50 IoU from 17.3 percent to 68.9 percent, roughly a four times gain.

TABLE VIII. BASELINE VERSUS OPTIMIZED DETECTOR

Metric	Baseline (Run 1)	Optimized (Run 2)	Change
Backbone	YOLOv8n (3.2M)	YOLOv8s (11.1M)	3.5x capacity
Resolution	640 x 640	800 x 800	56 pct more pixels
Epochs	15 (early stop)	50 (converged)	loss down over 65 pct
Precision	27.5 pct	77.6 pct	+50.1 points
Recall	21.7 pct	63.1 pct	+41.4 points
mAP50	17.3 pct	68.9 pct	about 4x
Vehicle mAP50	about 40.0 pct	80.7 pct	high reliability

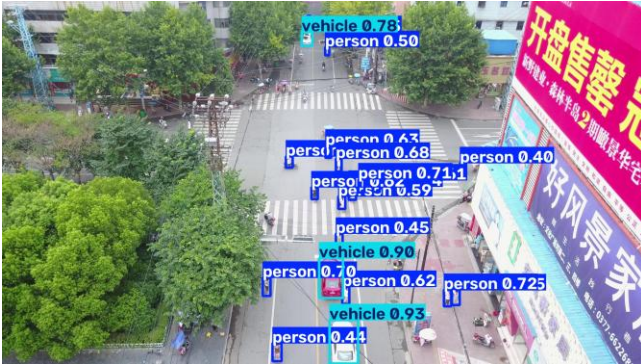


Fig. 6. Detector output on a held out VisDrone frame, showing person and vehicle boxes with confidence.

Taken together, the results support the two claims of this work. Coordinating the two drones with the Hungarian allocator roughly halves wasted coverage and speeds up exploration, and that better coverage carries through to finding more targets. The detector, fine tuned on VisDrone and reduced to the two classes that matter for rescue, localizes those targets on the shared map with an error smaller than one grid cell.

REFERENCES

- [1] B. Yamauchi, "A frontier-based approach for autonomous exploration," in Proc. IEEE Int. Symp. Computational Intelligence in Robotics and Automation (CIRA), 1997, pp. 146-151.
- [2] B. Yamauchi, "Frontier-based exploration using multiple robots," in Proc. 2nd Int. Conf. on Autonomous Agents, 1998, pp. 47-53.
- [3] W. Burgard, M. Moors, C. Stachniss, and F. Schneider, "Coordinated multi-robot exploration," IEEE Transactions on Robotics, vol. 21, no. 3, pp. 376-386, 2005.
- [4] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, "Receding horizon next-best-view planner for 3D exploration," in Proc. IEEE Int. Conf. on Robotics and Automation (ICRA), 2016, pp. 1462-1468.
- [5] C. Wang, H. Ma, W. Chen, L. Liu, and M. Q.-H. Meng, "Efficient autonomous exploration with incrementally built topological map in 3-D environments," IEEE Transactions on Instrumentation and Measurement, vol. 69, pp. 9853-9865, 2020.
- [6] P. Ghassemi and S. Chowdhury, "Multi-robot task allocation in disaster response: Addressing dynamic tasks with deadlines and robots with range and payload constraints," Robotics and Autonomous Systems, vol. 147, p. 103905, 2022.
- [7] M. Saboia et al., "ACHORD: Communication-aware multi-robot coordination with intermittent connectivity," IEEE Robotics and Automation Letters, vol. 7, no. 4, pp. 10184-10191, 2022.
- [8] L. Bartolomei, L. Teixeira, and M. Chli, "Fast multi-UAV decentralized exploration of forests," IEEE Robotics and Automation Letters, vol. 8, no. 9, pp. 5576-5583, 2023.

- [9] "Multi-robot cooperative autonomous exploration via task allocation in terrestrial environments," *Frontiers in Neurorobotics*, vol. 17, 1179033, 2023.
- [10] Q. Chen, R. Wang, M. Lyu, and J. Zhang, "Transformer-based reinforcement learning for multi-robot autonomous exploration," *Sensors*, vol. 24, no. 16, p. 5083, 2024.
- [11] T. Zhang, H. Shen, Y. Yin, J. Xu, J. Yu, and Y. Pan, "LECES: A low-bandwidth and efficient collaborative exploration system with distributed multi-UAV," *IEEE Robotics and Automation Letters*, vol. 9, no. 9, pp. 7795-7802, 2024.
- [12] G. Kucukayan and H. Karacan, "YOLO-IHD: Improved real-time human detection system for indoor drones," *Sensors*, vol. 24, no. 3, p. 922, 2024.
- [13] "Real-time search and rescue with drones: A deep learning approach for small-object detection based on YOLO," *Drones*, vol. 9, no. 8, p. 514, 2025.
- [14] X. Zhang, Y. Feng, N. Wang, G. Lu, and S. Mei, "Aerial person detection for search and rescue: Survey and benchmarks," *Journal of Remote Sensing*, vol. 5, p. 0474, 2025.
- [15] Y. Hui, G. Chang, J. Lu, X. Zhang, Z. Li, and B. Tian, "PC-Explorer: Decentralized multi-UAV exploration in bandwidth-limited environments," *IEEE Robotics and Automation Letters*, vol. 10, no. 9, pp. 9200-9207, 2025.
- [16] G. Lu, H. Ding, J. Hou, and S. Zheng, "Multi-robot collaborative exploration on communication-constrained environments," *IEEE Access*, vol. 13, pp. 31205-31214, 2025.
- [17] R. Wang, M. Lyu, and J. Zhang, "A multi-robot collaborative exploration method based on deep reinforcement learning and knowledge distillation," *Mathematics*, vol. 13, no. 1, p. 173, 2025.
- [18] "A systematic literature review on multi-robot task allocation," *ACM Computing Surveys*, 2025.
- [19] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Prentice Hall, 2015.
- [20] Centre for Research on the Epidemiology of Disasters (CRED), 2024 Disasters in Numbers (EM-DAT Annual Report), Brussels, 2025.
- [21] G. E. M. Abro, Z. A. Ali, and R. J. Masood, "Synergistic UAV motion: A comprehensive review on advancing multi-agent coordination," *ICCK Transactions on Sensing, Communication, and Control*, vol. 1, no. 2, pp. 72-88, 2024.
- [22] L. Yang, Y. Hao, J. Xu, and M. Li, "Multi-UAV collaborative target search method in unknown dynamic environment," *Sensors*, vol. 24, no. 23, p. 7639, 2024.